

**LAPORAN PRAKTIKUM**  
**PEMROGRAMAN BERORIENTASI OBJEK**  
**MODUL 11**



**Disusun Oleh:**

Silviana Maristha Ginting (105224001)

**PROGRAM STUDI ILMU KOMPUTER**  
**FAKULTAS SAINS DAN KOMPUTER**  
**UNIVERSITAS PERTAMINA**  
**2026**

## **Pendahuluan**

Error Handling merupakan mekanisme dalam bahasa pemrograman Java yang digunakan untuk menangani kesalahan (exception) yang terjadi selama program dijalankan. Dengan adanya Error Handling, program dapat tetap berjalan dengan baik tanpa mengalami crash ketika terjadi kesalahan input maupun kesalahan proses. Java menyediakan berbagai mekanisme penanganan exception seperti try-catch-finally, throw, throws, serta pembuatan custom exception sesuai kebutuhan aplikasi.

Pada praktikum ini dibuat program Sistem Reservasi Tiket Kereta Api "JAVA EXPRESS" menggunakan bahasa pemrograman Java dengan menerapkan konsep Pemrograman Berorientasi Objek (PBO) dan Error Handling. Program digunakan untuk menampilkan jadwal kereta, melakukan pemesanan tiket, melakukan validasi data penumpang, serta memeriksa ketersediaan kursi sebelum proses reservasi dilakukan.

Dalam implementasinya digunakan custom checked exception dan custom unchecked exception untuk menangani berbagai kondisi kesalahan yang mungkin terjadi selama program berjalan. Selain itu digunakan mekanisme try-catch-finally, throw, throws, serta InputMismatchException untuk memastikan program tetap stabil dan mampu memberikan pesan kesalahan yang sesuai kepada pengguna. Melalui praktikum ini diharapkan mahasiswa dapat memahami penerapan Error Handling dalam membangun aplikasi yang lebih aman, terstruktur, dan mudah dipelihara.

## Variabel

| No | Nama Variabel   | Tipe Data       | Fungsi                                             |
|----|-----------------|-----------------|----------------------------------------------------|
| 1  | kode            | String          | Menyimpan kode kereta yang dipilih pengguna.       |
| 2  | nama            | String          | Menyimpan nama penumpang.                          |
| 3  | nik             | String          | Menyimpan NIK penumpang.                           |
| 4  | jumlah          | int             | Menyimpan jumlah tiket yang dipesan.               |
| 5  | pilihan         | int             | Menyimpan pilihan menu utama.                      |
| 6  | jalan           | boolean         | Mengontrol perulangan program.                     |
| 7  | input           | Scanner         | Digunakan untuk membaca input dari keyboard.       |
| 8  | reservasi       | SistemReservasi | Objek utama yang menjalankan sistem reservasi.     |
| 9  | daftarKereta    | ArrayList       | Menyimpan seluruh data kereta yang tersedia.       |
| 10 | keretaDitemukan | KeretaApi       | Menyimpan objek kereta yang berhasil ditemukan.    |
| 11 | kodeKereta      | String          | Menyimpan kode kereta pada proses pemesanan tiket. |
| 12 | namaPenumpang   | String          | Menyimpan nama penumpang saat reservasi.           |

|    |             |           |                                                                  |
|----|-------------|-----------|------------------------------------------------------------------|
| 13 | jumlahTiket | int       | Menyimpan jumlah tiket yang akan dipesan.                        |
| 14 | namaKereta  | String    | Menyimpan nama kereta pada class TiketHabisException.            |
| 15 | sisakursi   | int       | Menyimpan jumlah kursi yang masih tersedia.                      |
| 16 | pesan       | String    | Menyimpan pesan kesalahan yang dikirim melalui exception.        |
| 17 | kereta      | KeretaApi | Variabel perulangan untuk mengakses data kereta dalam ArrayList. |
| 18 | c           | char      | Menyimpan karakter NIK yang sedang diperiksa saat validasi.      |

## Dokumentasi dan Pembahasan Code

### DataPenumpangTidakValidException.java

```
src > J DataPenumpangTidakValidException.java > DataPenumpangTidakValidException
1  public class DataPenumpangTidakValidException extends RuntimeException {
2
3  public DataPenumpangTidakValidException(String pesan) {
4      super(pesan);
5  }
6  }
```

### RuteTidakDitemukanException.java

```
src > J RuteTidakDitemukanException.java > RuteTidakDitemukanException
1  public class RuteTidakDitemukanException extends Exception {
2
3      public RuteTidakDitemukanException(String pesan) {
4          super(pesan);
5      }
6  }
```

### TiketHabisException.java

```
src > J TiketHabisException.java > ...
1  public class TiketHabisException extends Exception {
2
3      private String namaKereta;
4      private int sisaKursi;
5
6      public TiketHabisException(String namaKereta, int sisaKursi) {
7
8          super(message: "Tiket tidak mencukupi");
9
10         this.namaKereta = namaKereta;
11         this.sisaKursi = sisaKursi;
12     }
13
14     public String getNamaKereta() {
15         return namaKereta;
16     }
17
18     public int getSisaKursi() {
19         return sisaKursi;
20     }
21 }
22
23
```

## KeretaApi.java

```
src > J KeretaApi.java > KeretaApi > getSisaKursi()
1  public class KeretaApi {
2
3      private String kode;
4      private String nama;
5      private String rute;
6      private int sisaKursi;
7
8      public KeretaApi(String kode, String nama, String rute,int sisaKursi) {
9
10         this.kode = kode;
11         this.nama = nama;
12         this.rute = rute;
13         this.sisaKursi = sisaKursi;
14     }
15
16     public String getKode() {
17         return kode;
18     }
19
20     public String getNama() {
21         return nama;
22     }
23
24     public String getRute() {
25         return rute;
26     }
27
28     public int getSisaKursi() {
29         return sisaKursi;
30     }
31
32     public void kurangiKursi(int jumlah) {
33         sisaKursi -= jumlah;
34     }
35
36     @Override
37     public String toString() { return kode + " " + nama + " " + rute + " " + "Sisa Kursi: " + sisaKursi; }
38
39 }
```

## SistemReservasi.java

```
src > J SistemReservasi.java > SistemReservasi > pesanTiket(String, String, String, int)
1  import java.util.ArrayList;
2
3  public class SistemReservasi {
4
5      private ArrayList<KeretaApi> daftarKereta;
6
7      public SistemReservasi() {
8
9          daftarKereta = new ArrayList<>();
10
11          daftarKereta.add(new KeretaApi( kode: "K01", nama: "Argo Bromo", rute: "JKT - SBY",sisaKursi: 50));
12          daftarKereta.add(new KeretaApi( kode: "K02",nama: "Parahyangan",rute: "JKT - BDG",sisaKursi: 15));
13      }
14
15      public void tampilkanJadwal() {
16
17          System.out.println(x: "\n===== JADWAL KERETA =====");
18
19          for (KeretaApi kereta : daftarKereta) {
20              System.out.println(kereta);
21          }
22      }
23
24      public void pesanTiket(String kodeKereta,String nik,String namaPenumpang,int jumlahTiket)
25
26          throws RuteTidakDitemukanException,TiketHabisException {
27
28              // Validasi NIK
29              if (nik.length() != 16) {
30
31                  throw new DataPenumpangTidakValidException(pesan: "NIK harus 16 digit.");
32              }
33
34          }
```

```

35 // Validasi NIK harus angka
36 for (int i = 0; i < nik.length(); i++) {
37     char c = nik.charAt(i); // Ambil karakter per karakter
38
39     if (c < '0' || c > '9') {
40         throw new DataPenumpangTidakValidException(pesan: "NIK hanya boleh berisi angka.");
41     }
42
43 // Validasi jumlah tiket
44 if (jumlahTiket <= 0) {
45     throw new IllegalArgumentException(s: "Jumlah tiket harus lebih dari 0.");
46 }
47
48 KeretaApi keretaDitemukan = null;
49
50 for (KeretaApi kereta : daftarKereta) {
51     if (kereta.getKode().equalsIgnoreCase(kodeKereta)) {
52         keretaDitemukan = kereta;
53         break;
54     }
55 }
56 }

```

```

54 for (KeretaApi kereta : daftarKereta) {
55     if (kereta.getKode().equalsIgnoreCase(kodeKereta)) {
56         keretaDitemukan = kereta;
57         break;
58     }
59 }
60
61 if (keretaDitemukan == null) {
62     throw new RuteTidakDitemukanException(pesan: "Kode kereta tidak ditemukan.");
63 }
64
65 if (jumlahTiket > keretaDitemukan.getSisaKursi()) {
66     throw new TiketHabisException(keretaDitemukan.getNama(), keretaDitemukan.getSisaKursi());
67 }
68
69 keretaDitemukan.kurangiKursi(jumlahTiket);
70
71 System.out.println(x: "\n===== PEMESANAN BERHASIL =====");
72 System.out.println("Nama Penumpang : " + namaPenumpang);
73 System.out.println("NIK: " + nik);
74 System.out.println("Kereta: " + keretaDitemukan.getNama());
75 System.out.println("Jumlah Tiket: " + jumlahTiket);
76 System.out.println("Sisa Kursi: " + keretaDitemukan.getSisaKursi());
77 }
78 }
79

```

## Main.java

```
src > Main.java > Main > main(String[])
1  import java.util.InputMismatchException;
2  import java.util.Scanner;
3
4  public class Main {
5
6      Run | Debug
7      public static void main(String[] args) {
8
9          Scanner input = new Scanner(System.in);
10
11          SistemReservasi reservasi =
12              new SistemReservasi();
13
14          boolean jalan = true;
15
16          try {
17
18              while (jalan) {
19
20                  try {
21
22                      System.out.println(x: "\n=====");
23                      System.out.println(x: "    JAVA EXPRESS");
24                      System.out.println(x: "=====");
25                      System.out.println(x: "1. Lihat Jadwal");
26                      System.out.println(x: "2. Pesan Tiket");
27                      System.out.println(x: "3. Keluar");
28                      System.out.print(s: "Pilih Menu : ");
29
30                      int pilihan =input.nextInt();
31                      input.nextLine();
32                      switch (pilihan) {
33                          case 1:
34                              reservasi.tampilkanJadwal();
35                              break;
36                          case 2:
37                              System.out.print(s: "Kode Kereta : ");
38                              String kode =input.nextLine();
39                              System.out.print(s: "NIK : ");
40                              String nik =input.nextLine();
41                              System.out.print(s: "Nama Penumpang : ");
42                              String nama =input.nextLine();
43                              System.out.print(s: "Jumlah Tiket : ");
44                              int jumlah =input.nextInt();
45                              input.nextLine();
46                              reservasi.pesanTiket(kode, nik,nama,jumlah);
47                              break;
48                          case 3:
49                              System.out.println( x: "\nTerima kasih telah menggunakan JAVA EXPRESS.");
50                              jalan = false;
51                              break;
52                          default:
53                              System.out.println( x: "Menu tidak tersedia.");
54                      }
55                  } catch (InputMismatchException e) {
56                      System.out.println(x: "Input harus berupa angka!");
57                  }
58              }
59          }
60      }
```



```

58
59         input.nextLine();
60     }
61     catch (DataPenumpangTidakValidException e) {
62         System.out.println("Error Data Penumpang : " + e.getMessage());
63     }
64     catch (RuteTidakDitemukanException e) {
65         System.out.println("Error Rute : " + e.getMessage());
66     }
67     catch (TiketHabisException e) {
68         System.out.println("Tiket untuk " + e.getNamaKereta() + " tidak mencukupi.");
69         System.out.println("Sisa kursi tersedia : " + e.getSisaKursi());
70     }
71     catch (IllegalArgumentException e) {
72         System.out.println(e.getMessage());
73     }
74     catch (Exception e) {
75         System.out.println("Terjadi kesalahan : " + e.getMessage());
76     }
77 }
78
79 } finally {
80     input.close();
81     System.out.println("Scanner berhasil ditutup.");
82 }
83
84

```

## Penjelasan tiap class

### DataPenumpangTidakValidException.java

```
src > DataPenumpangTidakValidException.java > DataPenumpangTidakValidException
1 public class DataPenumpangTidakValidException extends RuntimeException {
2
3     public DataPenumpangTidakValidException(String pesan) {
4         super(pesan);
5     }
6 }
```

Class `DataPenumpangTidakValidException` digunakan sebagai custom unchecked exception yang berfungsi menangani kesalahan data penumpang. Exception ini akan muncul ketika NIK yang dimasukkan tidak sesuai ketentuan, yaitu tidak berjumlah 16 digit atau mengandung karakter selain angka.

### Constructor `DataPenumpangTidakValidException()`

Constructor digunakan untuk menerima pesan kesalahan yang dikirimkan saat exception dilempar menggunakan keyword `throw`. Pesan tersebut diteruskan ke superclass `RuntimeException` menggunakan method `super()` sehingga dapat ditampilkan pada blok `catch`.

### RuteTidakDitemukanException.java

```
src > RuteTidakDitemukanException.java > RuteTidakDitemukanException
1 public class RuteTidakDitemukanException extends Exception {
2
3     public RuteTidakDitemukanException(String pesan) {
4         super(pesan);
5     }
6 }
```

Class `RuteTidakDitemukanException` digunakan sebagai custom checked exception yang berfungsi menangani kesalahan ketika kode kereta yang dimasukkan pengguna tidak ditemukan pada sistem.

### Constructor `RuteTidakDitemukanException()`

Constructor digunakan untuk menerima pesan kesalahan dan meneruskannya ke superclass `Exception` menggunakan method `super()`. Karena merupakan checked exception, exception ini harus ditangani menggunakan `try-catch` atau dideklarasikan menggunakan `throws`.

## TiketHabisException.java

```
src > J TiketHabisException.java > ...
1  public class TiketHabisException extends Exception {
2
3      private String namaKereta;
4      private int sisaKursi;
5
6      public TiketHabisException(String namaKereta, int sisaKursi) {
7
8          super(message: "Tiket tidak mencukupi");
9
10         this.namaKereta = namaKereta;
11         this.sisaKursi = sisaKursi;
12     }
13
14     public String getNamaKereta() {
15         return namaKereta;
16     }
17
18     public int getSisaKursi() {
19         return sisaKursi;
20     }
21 }
22
23
```

Class TiketHabisException digunakan sebagai customer checked exception untuk menangani kondisi ketika jumlah tiket yang dipesan melebihi jumlah kursi yang tersedia.

### Constructor TiketHabisException()

Constructor digunakan untuk menyimpan informasi nama kereta dan jumlah kursi yang tersedia sehingga pesan error yang ditampilkan menjadi lebih informatif.

#### - Method getNamaKereta()

Method digunakan untuk mengambil nama kereta yang mengalami kekurangan kursi ketika exception terjadi.

#### - Method getSisaKursi()

Method digunakan untuk mengambil jumlah kursi yang masih tersedia sehingga dapat ditampilkan kepada pengguna.

## KeretaApi.java

```
src > KeretaApi.java > keretaApi > getSisaKursi
1 public class KeretaApi {
2
3     private String kode;
4     private String nama;
5     private String rute;
6     private int sisaKursi;
7
8     public KeretaApi(String kode, String nama, String rute, int sisaKursi) {
9
10        this.kode = kode;
11        this.nama = nama;
12        this.rute = rute;
13        this.sisaKursi = sisaKursi;
14    }
15
16    public String getKode() {
17        return kode;
18    }
19
20    public String getNama() {
21        return nama;
22    }
23
24    public String getRute() {
25        return rute;
26    }
27
28    public int getSisaKursi() {
29        return sisaKursi;
30    }
31
32    public void kurangiKursi(int jumlah) {
33        sisaKursi -= jumlah;
34    }
35
36    @Override
37    public String toString() { return kode + " " + nama + " " + rute + " " + "Sisa Kursi: " + sisaKursi; }
38
39 }
```

Class KeretaApi digunakan sebagai model data untuk menyimpan informasi kereta yang tersedia dalam sistem reservasi.

### Constructor KeretaApi()

Constructor digunakan untuk menginisialisasi atribut kode kereta, nama kereta, rute perjalanan, dan jumlah kursi yang tersedia ketika objek dibuat.

- **Method getKode()**

Method digunakan untuk mengambil kode kereta.

- **Method getNama()**

Method digunakan untuk mengambil nama kereta.

- **Method getRute()**

Method digunakan untuk mengambil informasi rute perjalanan kereta.

- **Method getSisaKursi()**

Method digunakan untuk mengambil jumlah kursi yang masih tersedia.

- **Method kurangiKursi()**

Method digunakan untuk mengurangi jumlah kursi setelah proses pemesanan berhasil dilakukan.

- **Method toString()**

Method digunakan untuk mengubah objek kereta menjadi bentuk teks sehingga dapat ditampilkan pada menu jadwal kereta.

## SistemReservasi.java

```
src > J SistemReservasi.java > SistemReservasi > pesanTiket(String, String, String, int)
1  import java.util.ArrayList;
2
3  public class SistemReservasi {
4
5      private ArrayList<KeretaApi> daftarKereta;
6
7      public SistemReservasi() {
8
9          daftarKereta = new ArrayList<>();
10
11          daftarKereta.add(new KeretaApi( kode: "K01", nama: "Argo Bromo", rute: "JKT - SBY", sisaKursi: 50));
12
13          daftarKereta.add(new KeretaApi( kode: "K02", nama: "Parahyangan", rute: "JKT - BDG", sisaKursi: 15));
14      }
15
16      public void tampilkanJadwal() {
17
18          System.out.println(x: "\n==== JADWAL KERETA =====");
19
20          for (KeretaApi kereta : daftarKereta) {
21              System.out.println(kereta);
22          }
23      }
24
25      public void pesanTiket(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket)
26
27          throws RuteTidakDitemukanException, TiketHabisException {
28
29          // Validasi NIK
30          if (nik.length() != 16) {
31
32              throw new DataPenumpangTidakValidException(pesan: "NIK harus 16 digit.");
33          }
34
35          // Validasi NIK harus angka
36          for (int i = 0; i < nik.length(); i++) {
37
38              char c = nik.charAt(i); // Ambil karakter per karakter
39
40              if (c < '0' || c > '9') {
41
42                  throw new DataPenumpangTidakValidException(pesan: "NIK hanya boleh berisi angka.");
43              }
44          }
45
46          // Validasi jumlah tiket
47          if (jumlahTiket <= 0) {
48
49              throw new IllegalArgumentException(s: "Jumlah tiket harus lebih dari 0.");
50          }
51
52          KeretaApi keretaDitemukan = null;
53
54          for (KeretaApi kereta : daftarKereta) {
55
56              if (kereta.getKode().equalsIgnoreCase(kodeKereta)) {
57
58                  keretaDitemukan = kereta;
59                  break;
60              }
61          }
62      }
63  }
```

```

53
54   for (KeretaApi kereta : daftarKereta) {
55
56       if (kereta.getKode().equalsIgnoreCase(kodeKereta)) {
57
58           keretaDitemukan = kereta;
59           break;
60       }
61   }
62
63   if (keretaDitemukan == null) {
64
65       throw new RuteTidakDitemukanException( pesan: "Kode kereta tidak ditemukan.");
66   }
67
68   if (jumlahTiket > keretaDitemukan.getSisaKursi()) {
69
70       throw new TiketHabisException(keretaDitemukan.getNama(), keretaDitemukan.getSisaKursi());
71   }
72
73   keretaDitemukan.kurangiKursi(jumlahTiket);
74
75   System.out.println(x: "\n===== PEMESANAN BERHASIL =====");
76   System.out.println("Nama Penumpang : " + namaPenumpang);
77   System.out.println("NIK: " + nik);
78   System.out.println("Kereta: " + keretaDitemukan.getNama());
79   System.out.println("Jumlah Tiket: " + jumlahTiket);
80   System.out.println("Sisa Kursi: " + keretaDitemukan.getSisaKursi());
81 }
82 }
83

```

Class SistemReservasi berfungsi sebagai pusat pengelolaan data kereta dan proses reservasi tiket.

### Constructor SistemReservasi()

Constructor digunakan untuk menginisialisasi ArrayList dan menambahkan data awal kereta yaitu K01 – Argo Bromo dan K02 – Parahyangan sesuai ketentuan soal.

#### - Method tampilkanJadwal()

Method digunakan untuk menampilkan seluruh jadwal kereta yang tersimpan pada ArrayList menggunakan perulangan for-each.

#### - Method pesanTiket()

Method digunakan untuk melakukan proses reservasi tiket. Method melakukan validasi panjang NIK, memeriksa setiap karakter NIK menggunakan perulangan for, memvalidasi jumlah tiket, mencari kereta berdasarkan kode, memeriksa ketersediaan kursi, kemudian mengurangi jumlah kursi apabila proses reservasi berhasil dilakukan.

## Main.java

```
src > Main.java > Main > main(String[])
1  import java.util.InputMismatchException;
2  import java.util.Scanner;
3
4  public class Main {
5
6      Run | Debug
      public static void main(String[] args) {
7
8          Scanner input = new Scanner(System.in);
9
10         SistemReservasi reservasi =
11             new SistemReservasi();
12
13         boolean jalan = true;
14
15         try {
16
17             while (jalan) {
18
19                 try {
20
21                     System.out.println(x: "\n=====");
22                     System.out.println(x: "    JAVA EXPRESS");
23                     System.out.println(x: "=====");
24                     System.out.println(x: "1. Lihat Jadwal");
25                     System.out.println(x: "2. Pesan Tiket");
26                     System.out.println(x: "3. Keluar");
27                     System.out.print(s: "Pilih Menu : ");
28
29                     int pilihan = input.nextInt();
30                     input.nextLine();
31                     switch (pilihan) {
32                         case 1:
33                             reservasi.tampilkanJadwal();
34                             break;
35                         case 2:
36                             System.out.print(s: "Kode Kereta : ");
37                             String kode = input.nextLine();
38                             System.out.print(s: "NIK : ");
39                             String nik = input.nextLine();
40                             System.out.print(s: "Nama Penumpang : ");
41                             String nama = input.nextLine();
42                             System.out.print(s: "Jumlah Tiket : ");
43                             int jumlah = input.nextInt();
44                             input.nextLine();
45                             reservasi.pesanTiket(kode, nik, nama, jumlah);
46                             break;
47                         case 3:
48                             System.out.println(x: "\nTerima kasih telah menggunakan JAVA EXPRESS.");
49                             jalan = false;
50                             break;
51                         default:
52                             System.out.println(x: "Menu tidak tersedia.");
53                     }
54                 }
55             }
56             catch (InputMismatchException e) {
57                 System.out.println(x: "Input harus berupa angka!");
58             }
59         }
60     }
61 }
```

```

58
59         input.nextLine();
60     }
61     catch (DataPenumpangTidakValidException e) {
62         System.out.println("Error Data Penumpang : " + e.getMessage());
63     }
64     catch (RuteTidakDitemukanException e) {
65         System.out.println("Error Rute : " + e.getMessage());
66     }
67     catch (TiketHabisException e) {
68         System.out.println("Tiket untuk " + e.getNamaKereta() + " tidak mencukupi.");
69         System.out.println("Sisa kursi tersedia : " + e.getSisaKursi());
70     }
71     catch (IllegalArgumentException e) {
72         System.out.println( e.getMessage());
73     }
74     catch (Exception e) {
75         System.out.println("Terjadi kesalahan : " + e.getMessage());
76     }
77 }
78
79 } finally {
80     input.close();
81     System.out.println( x: "\nScanner berhasil ditutup.");
82 }
83
84

```

Class Main merupakan class utama yang berfungsi sebagai titik awal eksekusi program Sistem Reservasi Tiket Kereta Api "JAVA EXPRESS".

### Method main()

Method main() digunakan untuk menjalankan seluruh fitur yang tersedia pada sistem. Program menampilkan menu utama, menerima input pengguna, memanggil method yang diperlukan pada class SistemReservasi, serta menangani berbagai exception menggunakan blok try-catch-finally. Selain itu method ini juga memastikan objek Scanner ditutup dengan benar menggunakan blok finally ketika program selesai dijalankan.



#### **IV. Kesimpulan**

Berdasarkan hasil praktikum yang telah dilakukan, program Sistem Reservasi Tiket Kereta Api "JAVA EXPRESS" berhasil dibuat menggunakan bahasa pemrograman Java dengan menerapkan konsep Pemrograman Berorientasi Objek (PBO) dan Error Handling. Program mampu menampilkan jadwal kereta, melakukan pemesanan tiket, melakukan validasi data penumpang, serta memeriksa ketersediaan kursi sebelum proses reservasi dilakukan.

Penerapan custom checked exception dan custom unchecked exception memungkinkan sistem menangani berbagai kondisi kesalahan secara lebih spesifik dan terstruktur. Selain itu, penggunaan keyword `throw`, `throws`, serta blok `try-catch-finally` membantu program dalam menangani error tanpa menyebabkan program berhenti secara tiba-tiba.

Melalui praktikum ini dapat dipahami bahwa Error Handling memiliki peran penting dalam meningkatkan keandalan dan stabilitas program. Dengan penanganan exception yang baik, program menjadi lebih aman, mudah dipelihara, serta mampu memberikan informasi kesalahan yang lebih jelas kepada pengguna.

## **v. Daftar Pustaka**

Tim Asisten Laboratorium Komputer. (2026). Modul Praktikum Pemrograman Berorientasi (PBO)

Modul 11 Program Studi Ilmu Komputer, Universitas Pertamina